

CFG

<Program> → <StatementList>

<StatementList> → <Statement> <StatementList> | ε

<Statement> → <Declaration>
 | <Assignment>
 | <Operation>
 | <Control>

<Declaration> → 'item' IDENTIFIER '=' NUMBER UNIT ';' | 'stat' IDENTIFIER '=' NUMBER UNIT ';'

<Assignment> → IDENTIFIER '=' <Expr> UNIT ';'

<Expr> → <Term> <ExprTail>

<ExprTail> → '+' <Term> <ExprTail>
 | '-' <Term> <ExprTail>
 | ε

<Term> → <Factor> <TermTail>

<TermTail> → '*' <Factor> <TermTail>
 | '/' <Factor> <TermTail>
 | ε

<Factor> → NUMBER
 | IDENTIFIER
 | '(' <Expr> ')'

<Operation> → 'narrate' STRING ';' | 'show' IDENTIFIER ';' | 'combine' IDENTIFIER 'with' IDENTIFIER ';' | 'equip' IDENTIFIER 'to' <Expr> UNIT ';' | 'rest' <Expr> UNIT ';'

<Control> → <LoopStmt>
 | <IfStmt>

<LoopStmt> → 'loop' NUMBER 'iterations' '{' <StatementList> '}'

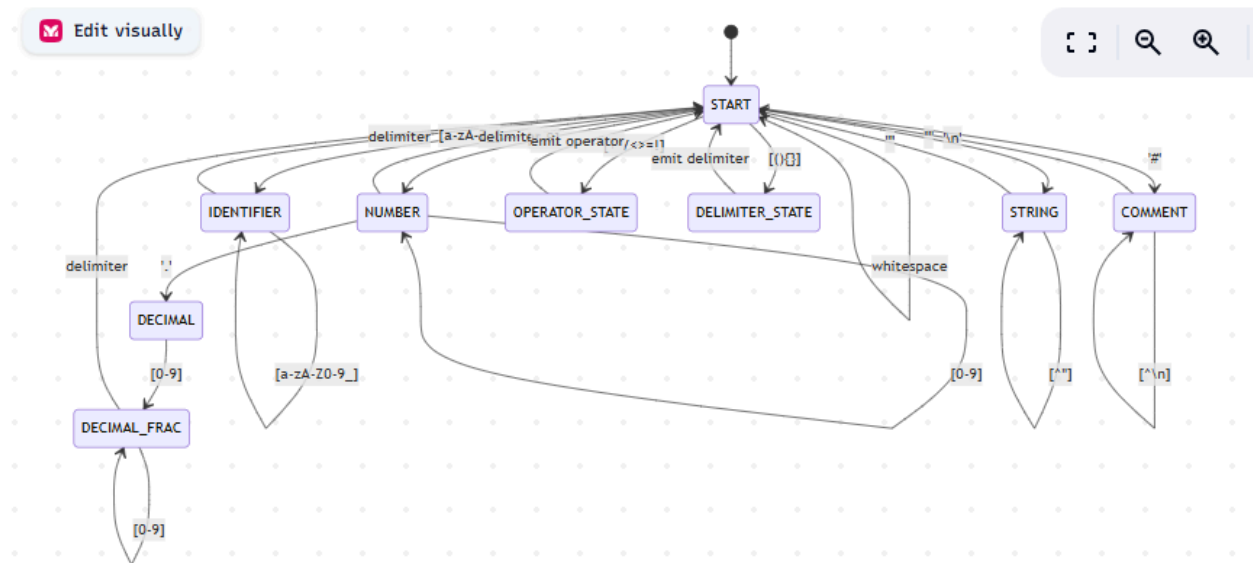
$\langle \text{IfStmt} \rangle \rightarrow \text{'if' } \langle \text{Condition} \rangle \text{'then' '{' } \langle \text{StatementList} \rangle \text{'}' } \langle \text{ElsePart} \rangle$

$\langle \text{ElsePart} \rangle \rightarrow \text{'else' '{' } \langle \text{StatementList} \rangle \text{'}' }$
 $\quad \mid \varepsilon$

$\langle \text{Condition} \rangle \rightarrow \langle \text{Expr} \rangle \langle \text{CompOp} \rangle \langle \text{Expr} \rangle$

$\langle \text{CompOp} \rangle \rightarrow \text{'=='}$
 $\quad \mid \text{'!='}$
 $\quad \mid \text{'>'}$
 $\quad \mid \text{'<'}$
 $\quad \mid \text{'>='}$
 $\quad \mid \text{'<='}$

DFA



stateDiagram-v2

[*] --> START

START --> IDENTIFIER: [a-zA-Z_]

IDENTIFIER --> IDENTIFIER: [a-zA-Z0-9_]

IDENTIFIER --> START: delimiter

START --> NUMBER: [0-9]

NUMBER --> NUMBER: [0-9]

NUMBER --> DECIMAL: '.'

NUMBER --> START: delimiter

DECIMAL --> DECIMAL_FRAC: [0-9]

DECIMAL_FRAC --> DECIMAL_FRAC: [0-9]

DECIMAL_FRAC --> START: delimiter

START --> STRING: '"'

STRING --> STRING: [^"]

STRING --> START: '''

START --> OPERATOR_STATE: [+*/<>=!]

OPERATOR_STATE --> START: emit operator

START --> DELIMITER_STATE: [() { }]

DELIMITER_STATE --> START: emit delimiter

START --> COMMENT: '#'

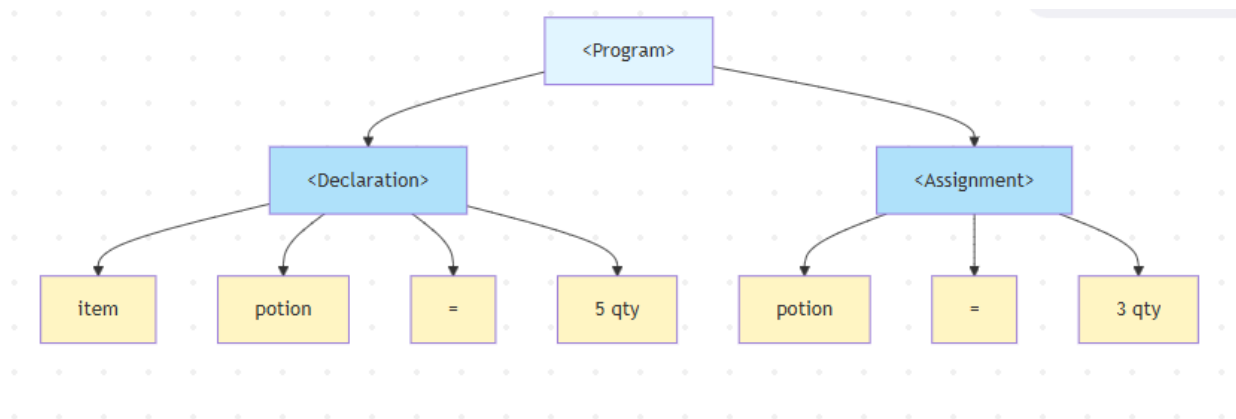
COMMENT --> COMMENT: [^\n]

COMMENT --> START: '\n'

START --> START: whitespace

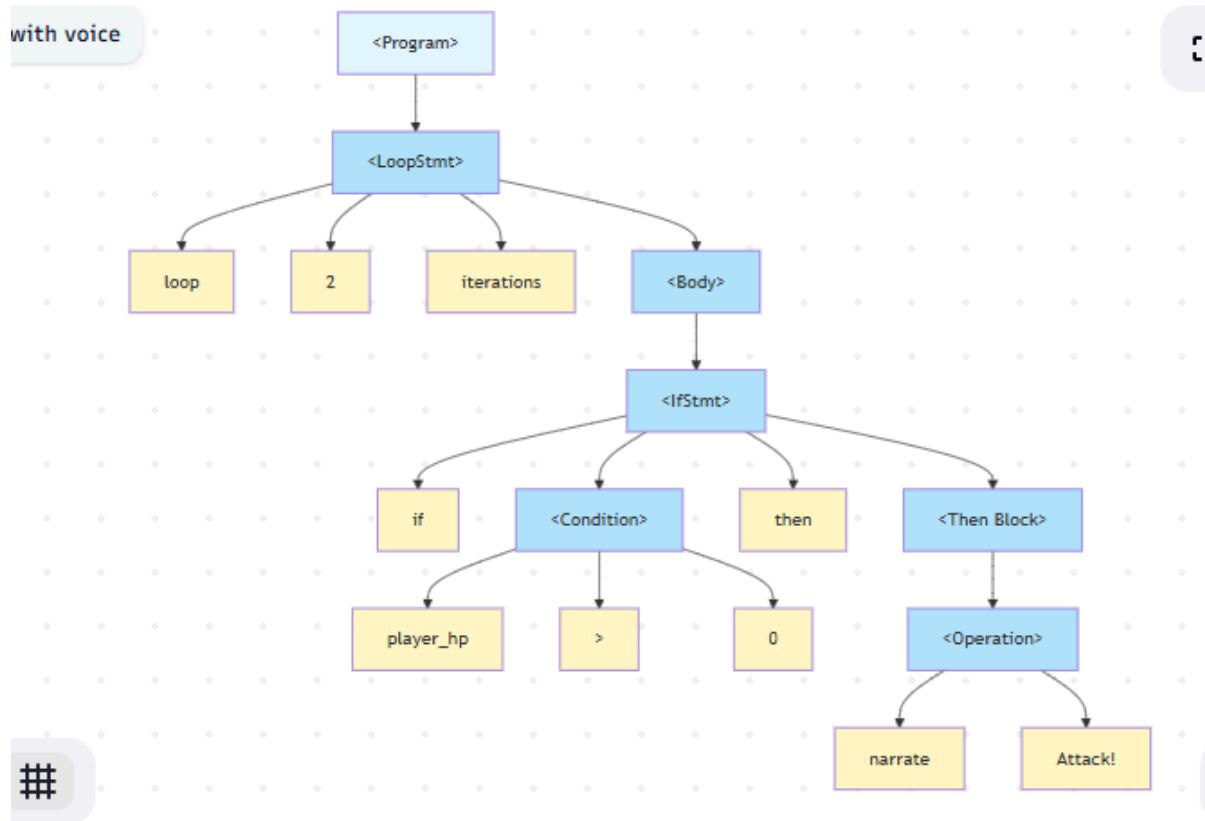
Parse Tree 1:

item potion = 5 qty;
potion = 3 qty;

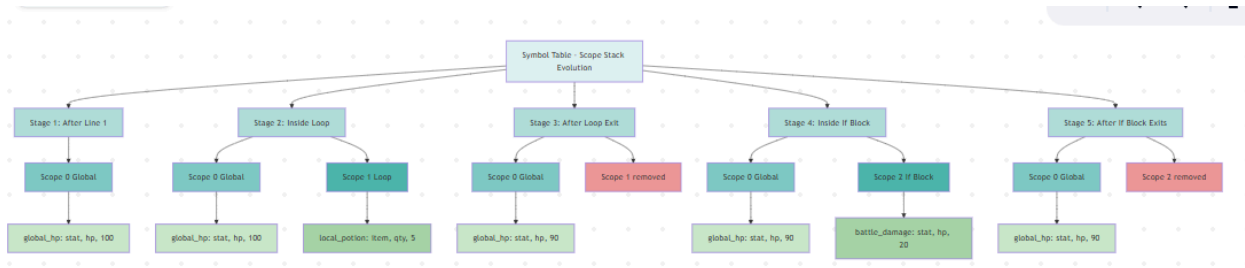


Parse Tree 2:

loop 2 iterations {
 if player_hp > 0 then {
 narrate "Attack!"; }
}



Symbol table



3.4 Complete Symbol Table Reference (Filled Table)

The filled symbol table is written in the same stage-by-stage format as the detailed diagram so the scope changes are easy to verify.

Stage 1: After Line 1 (Global Scope Only)

Name	Type	Unit	Value	Line Declared	Scope Level	Status
global_hp	stat	hp	100	1	0 (Global)	Active

Stage 2: Inside Loop (Two Scopes)

Name	Type	Unit	Value	Line Declared	Scope Level	Status
global_hp	stat	hp	100	1	0 (Global)	Active
local_potion	item	qty	5	4	1 (Loop)	Active

Stage 3: After Loop Exit (Global Scope Only, Updated Value)

Name	Type	Unit	Value	Line Declared	Scope Level	Status
------	------	------	-------	---------------	-------------	--------

global_hp	stat	hp	90	1	0 (Global)	Active
-----------	------	----	----	---	------------	--------

Stage 4: Inside If Block (Two Scopes)

Name	Type	Unit	Value	Line Declared	Scope Level	Status
global_hp	stat	hp	90	1	0 (Global)	Active
battle_damage	stat	hp	20	8	2 (If block)	Active

Stage 5: After If Block Exits (Global Scope Only)

Name	Type	Unit	Value	Line Declared	Scope Level	Status
global_hp	stat	hp	90	1	0 (Global)	Active

The two symbol tables are for **different purposes**, but they are based on the **same program**.

Difference:

- **Symbol table with scope visualization:** shows the **structure of scopes** and how they open/close.
- **Filled symbol table reference:** shows the **actual data inside the table** such as **Name**, **Type**, **Unit**, **Value**, **Line**, **Scope Level**, and **Status**.

So:

- The **diagram/table from the .mmd file** is for showing **scope flow**.
- The **filled table in the document** is for showing **exact symbol entries**.

Is it required to make two different tables?

- **Not two completely different symbol tables.**
- But it is good to show them in **two forms**:
 1. **Scope visualization:** how scopes change
 2. **Filled table:** what variables are stored in each scope

In short, you can think of them like this:

- **Visualization = how the symbol table behaves**
- **Filled table = what the symbol table contains**

For your submission, that is usually enough and looks complete. You do **not** need to invent a second unrelated symbol table. You just need:

- one **scope diagram**
- one **filled symbol table example** in stage-by-stage form

If you want, I can also give you a **very short 2-3 line version** you can paste into your submission document.